

DeepSeek Orientation: Requests, Execution, and Technology Lineage

Deep Dive into DeepSeek — Chapter 1

Yin Yang

Foundation Books Project

The one rule of this chapter

Identify the **changed object** before treating two results as comparable or attributing their difference to the model.

- A shared family name and a chat prompt are not enough: the message path depends on a template and tokenizer, a selected artifact and runtime, and decoding controls.
- Choosing between releases, assessing an improvement, and reproducing a result all require knowing what changed.

The one rule of this chapter

Identify the **changed object** before treating two results as comparable or attributing their difference to the model.

- A shared family name and a chat prompt are not enough: the message path depends on a template and tokenizer, a selected artifact and runtime, and decoding controls.
- Choosing between releases, assessing an improvement, and reproducing a result all require knowing what changed.

Goal: use one simple DeepSeek-V3 request to name every object on that path, then fix the decoder notation, number-format vocabulary, active-work accounting, and lineage distinctions the rest of the book relies on.

1. The request packet

1. The request packet
2. Tokens, decoding controls, and context

1. The request packet
2. Tokens, decoding controls, and context
3. The decoder path

1. The request packet
2. Tokens, decoding controls, and context
3. The decoder path
4. Numbers, precision, and active work

1. The request packet
2. Tokens, decoding controls, and context
3. The decoder path
4. Numbers, precision, and active work
5. Lineage and branches

The request packet

One request, five handoffs

System: *Use concise answers.* **User:** *Suggest a name for a small orange cat.*

request fields

One request, five handoffs

System: *Use concise answers.* **User:** *Suggest a name for a small orange cat.*



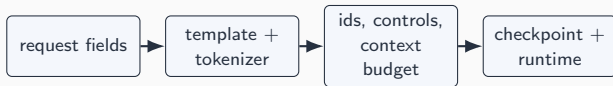
One request, five handoffs

System: *Use concise answers.* **User:** *Suggest a name for a small orange cat.*



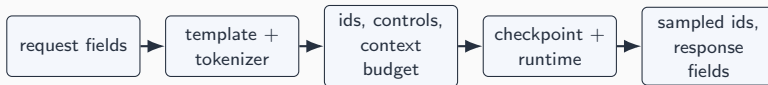
One request, five handoffs

System: *Use concise answers.* **User:** *Suggest a name for a small orange cat.*



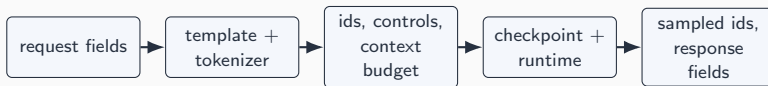
One request, five handoffs

System: *Use concise answers.* **User:** *Suggest a name for a small orange cat.*



One request, five handoffs

System: *Use concise answers.* **User:** *Suggest a name for a small orange cat.*



The *request packet*: visible messages, rendered/tokenized input, artifact identity, runtime state, generation controls, returned fields — a bookkeeping term, not a DeepSeek API field. This chapter has no pinned execution: the cat's name stays open.

Five artifacts, one lifecycle

Component	Compatibility evidence
Checkpoint weights	Tensor names, shapes, shard inventory match the loader.
Configuration	Allocations agree with checkpoint shapes and model class.
Tokenizer/template	Vocabulary, special tokens, revision recorded together.
Runtime	Formats, shapes, controls validated on the named path.
Interface	Local or hosted schema and its dated behavior recorded.

A valid artifact can still be incompatible with the rest. Code, weights, and services are separate publication surfaces: licenses, defaults, and limits attach to the named surface, not the family name. Pretraining and post-training change parameters; inference and serving change request-local and operational state around fixed weights.

Tokens, decoding controls, and context

From rendering to a chosen id

$$p_T(v) = \frac{\exp(z_v/T)}{\sum_{u \in \mathcal{V}} \exp(z_u/T)}$$

- Greedy: max logit. Top- p : smallest prefix reaching cumulative probability p , renormalized.
- Logits $(2, 1, 0)$ at $T = 1$: $\approx (0.665, 0.245, 0.090)$. Top- $p = 0.8$ keeps A, B :
 $\approx (0.731, 0.269)$.

“Token” hides at least four objects: token id, logical position, residual row, and attention state each name a different coordinate on the request path.

Context is a position budget; state is a stored object

$$L_p + L_{\text{gen}} \leq L_{\text{ctx}}$$

A documented 128K context, a 90,020-token prompt, and a 4,000-token reserve: 94,020 potential positions.

That arithmetic says nothing about cache bytes. The recorded V3 implementation exposes two shapes under the umbrella term *KV cache*: materialized `k_cache/v_cache`, or absorbed `kv_cache/pe_cache`. Both support causal reuse; their byte counts differ.

The decoder path

One block, two residual updates

$$H'_l = H_l + \text{Attention}_l(\text{Norm}_l(H_l)), \quad H_{l+1} = H'_l + \text{FFN}_l(\text{Norm}'_l(H'_l)).$$

- RMSNorm: $\text{RMSNorm}(x) = g \odot x / \text{RMS}(x)$ — controls magnitude, preserves direction.

One block, two residual updates

$$H'_l = H_l + \text{Attention}_l(\text{Norm}_l(H_l)), \quad H_{l+1} = H'_l + \text{FFN}_l(\text{Norm}'_l(H'_l)).$$

- RMSNorm: $\text{RMSNorm}(x) = g \odot x / \text{RMS}(x)$ — controls magnitude, preserves direction.
- Causal mask $M_{t,s} = 0$ for $s \leq t$, $-\infty$ for $s > t$: query row t reads only its own position and earlier ones.

One block, two residual updates

$$H'_l = H_l + \text{Attention}_l(\text{Norm}_l(H_l)), \quad H_{l+1} = H'_l + \text{FFN}_l(\text{Norm}'_l(H'_l)).$$

- RMSNorm: $\text{RMSNorm}(x) = g \odot x / \text{RMS}(x)$ — controls magnitude, preserves direction.
- Causal mask $M_{t,s} = 0$ for $s \leq t$, $-\infty$ for $s > t$: query row t reads only its own position and earlier ones.
- $\text{MHA}(\hat{H}) = \text{Concat}(O_1, \dots, O_{H_Q})W^O$ — heads project differently, then return to d_{model} .

One block, two residual updates

$$H'_l = H_l + \text{Attention}_l(\text{Norm}_l(H_l)), \quad H_{l+1} = H'_l + \text{FFN}_l(\text{Norm}'_l(H'_l)).$$

- RMSNorm: $\text{RMSNorm}(x) = g \odot x / \text{RMS}(x)$ — controls magnitude, preserves direction.
- Causal mask $M_{t,s} = 0$ for $s \leq t$, $-\infty$ for $s > t$: query row t reads only its own position and earlier ones.
- $\text{MHA}(\hat{H}) = \text{Concat}(O_1, \dots, O_{H_Q})W^O$ — heads project differently, then return to d_{model} .

DeepSeek-V3 applies RoPE to only part of Q, K — the MLA chapter derives that split.

Gated feed-forward, and the shape invariant

$$\text{GatedFFN}(x) = [\phi(xW_g) \odot (xW_u)] W_d, \quad \phi = \text{SiLU (inspected)}.$$

Invariant (Residual-shape preservation)

At every layer, $H_l, H'_l, H_{l+1} \in \mathbb{R}^{T \times d_{\text{model}}}$.

A two-position hand trace: attention residual $\approx (2, 1.609, 2, -2)$; after the gated update, $\approx (2.812, 2.226, 1.188, -2.617)$; teaching logits $(1.224, 0.969, 1.139)$.

A mechanism may change internal projections, active parameters, or stored state — MLA changes exactly what is retained — but the residual width and causal transition never bend.

Numbers, precision, and active work

A precision path has five stages, not one label

Name	Width	First reading
FP32	32 b	Wide range, fine spacing; accumulation/reference.

$$\hat{x} = sq, \quad q = 7, s = 0.25 \rightarrow 1.75; \quad q = 7, s = 0.50 \rightarrow 3.50.$$

A precision path has five stages, not one label

Name	Width	First reading
FP32	32 b	Wide range, fine spacing; accumulation/reference.
BF16	16 b	FP32-like range, coarser spacing than FP16.

$$\hat{x} = sq, \quad q = 7, s = 0.25 \rightarrow 1.75; \quad q = 7, s = 0.50 \rightarrow 3.50.$$

A precision path has five stages, not one label

Name	Width	First reading
FP32	32 b	Wide range, fine spacing; accumulation/reference.
BF16	16 b	FP32-like range, coarser spacing than FP16.
FP8	8 b	E4M3/E5M2; scale, accumulator still need naming.

$$\hat{x} = sq, \quad q = 7, s = 0.25 \rightarrow 1.75; \quad q = 7, s = 0.50 \rightarrow 3.50.$$

A precision path has five stages, not one label

Name	Width	First reading
FP32	32 b	Wide range, fine spacing; accumulation/reference.
BF16	16 b	FP32-like range, coarser spacing than FP16.
FP8	8 b	E4M3/E5M2; scale, accumulator still need naming.

$$\hat{x} = sq, \quad q = 7, \quad s = 0.25 \rightarrow 1.75; \quad q = 7, \quad s = 0.50 \rightarrow 3.50.$$

Payload + scale + multiplication + accumulation + output is the precision path. A label like FP8 names only the first stage.

Dense, routed, and a parameter ledger

Dense computation applies one subnetwork to every token; routed computation selects a token-specific subset and combines outputs.

$$\rho_{\text{active}} = \frac{37}{671} \approx 5.5\%$$

[REPORTED: DeepSeek-V3 repository]

$P_{\text{active}} = P_0 + P_s + \sum_j P_{e_j}$ under the release's counting convention. The 37B figure is a parameter ledger — FLOPs, traffic, and latency are separate, derived facts.

Lineage and branches

Six edge types, one architecture chain

DeepSeekMoE $\longrightarrow$ DeepSeek-V2 $\longrightarrow$ DeepSeek-V3

- **Architecture:** DeepSeekMoE, V2, V3, V4.

Six edge types, one architecture chain

DeepSeekMoE $\longrightarrow$ DeepSeek-V2 $\longrightarrow$ DeepSeek-V3

- **Architecture:** DeepSeekMoE, V2, V3, V4.
- **Training:** V3-Base $\rightarrow$ R1-Zero $\rightarrow$ R1.

Six edge types, one architecture chain

DeepSeekMoE $\longrightarrow$ DeepSeek-V2 $\longrightarrow$ DeepSeek-V3

- **Architecture:** DeepSeekMoE, V2, V3, V4.
- **Training:** V3-Base $\rightarrow$ R1-Zero $\rightarrow$ R1.
- **Distillation:** R1-generated samples $\rightarrow$ Qwen/Llama learners.

Six edge types, one architecture chain

DeepSeekMoE $\longrightarrow$ DeepSeek-V2 $\longrightarrow$ DeepSeek-V3

- **Architecture:** DeepSeekMoE, V2, V3, V4.
- **Training:** V3-Base $\rightarrow$ R1-Zero $\rightarrow$ R1.
- **Distillation:** R1-generated samples $\rightarrow$ Qwen/Llama learners.
- **Specialization:** Coder, Math/Prover, VL/Janus, OCR.

Six edge types, one architecture chain

DeepSeekMoE $\longrightarrow$ DeepSeek-V2 $\longrightarrow$ DeepSeek-V3

- **Architecture:** DeepSeekMoE, V2, V3, V4.
- **Training:** V3-Base $\rightarrow$ R1-Zero $\rightarrow$ R1.
- **Distillation:** R1-generated samples $\rightarrow$ Qwen/Llama learners.
- **Specialization:** Coder, Math/Prover, VL/Janus, OCR.
- **Adjacent research:** Engram — not identified in released V4 artifacts.

Six edge types, one architecture chain

DeepSeekMoE $\longrightarrow$ DeepSeek-V2 $\longrightarrow$ DeepSeek-V3

- **Architecture:** DeepSeekMoE, V2, V3, V4.
- **Training:** V3-Base $\rightarrow$ R1-Zero $\rightarrow$ R1.
- **Distillation:** R1-generated samples $\rightarrow$ Qwen/Llama learners.
- **Specialization:** Coder, Math/Prover, VL/Janus, OCR.
- **Adjacent research:** Engram — not identified in released V4 artifacts.
- **Systems realization:** FlashMLA, DeepGEMM, DeepEP, DualPipe, 3FS.

Six edge types, one architecture chain

DeepSeekMoE $\longrightarrow$ DeepSeek-V2 $\longrightarrow$ DeepSeek-V3

- **Architecture:** DeepSeekMoE, V2, V3, V4.
- **Training:** V3-Base $\rightarrow$ R1-Zero $\rightarrow$ R1.
- **Distillation:** R1-generated samples $\rightarrow$ Qwen/Llama learners.
- **Specialization:** Coder, Math/Prover, VL/Janus, OCR.
- **Adjacent research:** Engram — not identified in released V4 artifacts.
- **Systems realization:** FlashMLA, DeepGEMM, DeepEP, DualPipe, 3FS.

The endpoint objects decide the edge type; chronology alone never does.

From concept to code: one config, several threads

```
from DeepSeek-V3/inference/model.py

class ModelArgs:
    dim: int = 2048
    n_routed_experts: int = 64
    n_activated_experts: int = 6
    kv_lora_rank: int = 512
    qk_rope_head_dim: int = 64
```


From concept to code: one config, several threads

```
from DeepSeek-V3/inference/model.py

class ModelArgs:
    dim: int = 2048
    n_routed_experts: int = 64
    n_activated_experts: int = 6
    kv_lora_rank: int = 512
    qk_rope_head_dim: int = 64
```

One class, three of this chapter's threads: `n_routed/n_activated_experts` fix the dense/routed accounting; `kv_lora_rank` and `qk_rope_head_dim` preview the latent-plus-RoPE split the MLA chapter derives; every field here is a configuration commitment a checkpoint's tensors must match. [INSPECTED: DeepSeek-V3 inference/model.py]

Concept-to-code map for the chapter

- **checkpoint/config binding** → `DeepSeek-V3/inference/model.py`
- **released checkpoint layout** → `DeepSeek-V3/README_WEIGHTS.md`
- **two KV-cache shapes** → `DeepSeek-V3/inference/model.py`
- **RMSNorm and gated FFN** → `DeepSeek-V3/inference/model.py`

Name the object. Attach evidence to it. Only then compare.

Next: Chapter 2, Transformer Inference Cost and Measurement Foundations, turns these object identities into a byte- and FLOP-accounted cost model.